

Recipes & Ingredients

Company:- Amazon

Round :- In-person interviews at Amazon office.

Year:- 2026

Difficulty:- Medium to Hard

Topic:- Graphs, BFS, Topological Sort (Kahn's Algorithm), Queue, HashMap, and Dependency Management.

Source:- Desi Qna

QUESTION : RECIPES AND INGREDIENTS

Problem Description :-

You are given a list of recipes, the ingredients required for each recipe, and a list of supplies that are already available.

The important point is that a recipe can itself become an ingredient for another recipe.

Your task is to find all the recipes that can be prepared using the available supplies and the recipes that can be prepared from them.

Input Format :-

1. A list of recipes.
2. A list of ingredients required for each recipe.
3. A list of supplies that are already available.

Output Format :-

Return the list of all recipes that can be prepared.

Example :-

Recipes:

bread

sandwich

Ingredients:

bread - flour, water

sandwich - bread, vegetable

Supplies:

flour, water, vegetable

Sample Output :-

bread, sandwich

Explanation :-

We already have flour, water, and vegetable.

To make bread, we need flour and water. Both are available, so we can prepare bread.
Now, something interesting happens: bread itself becomes available as an ingredient.
To make sandwich we need bread and vegetable. We now have both, so we can prepare sandwich.
Therefore, the final answer is:
bread, sandwich

Approach to Solve This Problem :-

1) Understand the dependency between ingredients and recipes.

Think of every recipe as depending on its ingredients.

For example, bread depends on flour and water. Similarly, sandwich depends on bread and vegetable.

So, we need to keep track of which recipes are waiting for which ingredients.

2) Store how many ingredients each recipe needs.

For every recipe, we count the number of ingredients that are still required.

For example, bread needs:

flour + water

So initially, bread has 2 required ingredients.

Sandwich needs:

bread + vegetable

So sandwich also has 2 required ingredients.

3) Put all available supplies into a queue.

The supplies are ingredients that we already have.

In our example:

flour

water

vegetable

are already available, so we put them into a queue.

We process these available items one by one.

4) Check which recipes need the available ingredient.

Suppose we take flour from the queue.

We check which recipe needs flour.

We find that bread needs flour, so we reduce the number of ingredients still needed by bread.

Bread now needs only:

water

When we process water, bread will have no ingredients left to obtain.

5) Prepare the recipe when all its ingredients are available.

When the number of remaining ingredients becomes zero, it means we have everything required to prepare that recipe.

For bread:

flour ✓

water ✓

Therefore, bread can be prepared.

We add **bread** to our answer.

6) Treat the prepared recipe as a new ingredient.

This is the most important part of the problem.

Once we prepare bread, bread itself becomes available.

So we put bread into the queue.

Now when we process bread, we discover that sandwich needs bread.

Therefore, the number of ingredients still needed for sandwich decreases.

7) Continue the same process.

We continue processing the queue.

When vegetable is processed, sandwich now has:

bread ✓

vegetable ✓

So sandwich can also be prepared.

We add sandwich to our answer.

Finally, when there is nothing more to process, we return all the recipes we were able to prepare.

Why Do We Use BFS / Queue?

We use a queue because this problem can be treated like a dependency graph.

Think of it like this:

flour → bread

water → bread

bread → sandwich

vegetable → sandwich

When an ingredient becomes available, we move forward and check which recipes can now be completed.

This is similar to BFS (Breadth-First Search) and Topological Sort.

Java Solution :-

```
ution.java > *% Solution
import java.util.*;

class Solution {

    public List<String> findAllRecipes(
        String[] recipes,
        List<List<String>> ingredients,
        String[] supplies) {

        Map<String, List<String>> graph = new HashMap<>();
        Map<String, Integer> indegree = new HashMap<>();

        Queue<String> queue = new LinkedList<>();

        // Add all available supplies to the queue
        for (String supply : supplies) {
            queue.offer(supply);
        }

        // Build the graph and store dependencies
        for (int i = 0; i < recipes.length; i++) {

            String recipe = recipes[i];

            indegree.put(
                recipe,
                ingredients.get(i).size()
            );

            for (String ingredient : ingredients.get(i)) {

                graph.computeIfAbsent(
                    ingredient,
                    k -> new ArrayList<>()
                ).add(recipe);
            }
        }
    }
}
```

```
        List<String> result = new ArrayList<>();

        // Process available ingredients
        while (!queue.isEmpty()) {

            String item = queue.poll();

            if (!graph.containsKey(item)) {
                continue;
            }

            // Check recipes that need this ingredient
            for (String recipe : graph.get(item)) {

                indegree.put(
                    recipe,
                    indegree.get(recipe) - 1
                );

                // All ingredients are available
                if (indegree.get(recipe) == 0) {

                    result.add(recipe);

                    // Recipe can now act as an ingredient
                    queue.offer(recipe);
                }
            }
        }

        return result;
    }
}
```

Explanation:-

1. Create the graph.

```
Map<String, List<String>> graph = new HashMap<>();
```

The graph tells us which recipes need each ingredient.

For example:

flour → bread

water → bread

bread → sandwich

So, whenever flour becomes available, we immediately know that bread may be affected.

2. Store the number of required ingredients.

```
Map<String, Integer> indegree = new HashMap<>();
```

This stores how many ingredients each recipe still needs.

For example:

bread → 2

sandwich → 2

When flour becomes available, bread's count becomes 1.

When water becomes available, it becomes 0.

3. Add supplies to the queue.

```
for (String supply : supplies) {  
    queue.offer(supply);  
}
```

The supplies are already available, so we put them into the queue.

For our example:

flour

water

vegetable

4. Build the graph.

```
for (String ingredient : ingredients.get(i)) {  
    graph.computeIfAbsent(  
        ingredient,  
        k -> new ArrayList<>()  
    ).add(recipe);  
}
```

This connects every ingredient to the recipe that requires it.

For example:

flour → bread

water → bread

bread → sandwich

vegetable → sandwich

5. Take an available item from the queue.

String item = queue.poll();

We take one available ingredient from the queue.

For example, suppose we take:

flour

Now we check which recipes need flour.

6. Reduce the dependency count.

```
indegree.put(  
    recipe,  
    indegree.get(recipe) - 1  
);
```

Since flour is now available, bread has one less ingredient to wait for.

Initially:

bread → 2

After flour:

bread → 1

After water:

bread → 0

7. Prepare the recipe when the count becomes zero.

```
if (indegree.get(recipe) == 0) {  
    result.add(recipe);  
    queue.offer(recipe);  
}
```

When the count becomes zero, it means all ingredients are available.

So we add the recipe to our answer.

We also put the recipe into the queue because it can now be used as an ingredient for another recipe.

For example:

bread → sandwich

8. Return the final answer.

return result;

After processing everything, the result contains all recipes that can be prepared.

For our example:

[bread, sandwich]

Complexity Analysis :-

Time Complexity: $O(V + E)$

Here, V represents the recipes and ingredients, while E represents the dependency relationships between them.